

# Introduction

Structural design patterns are concerned with the **composition of classes and objects**. They help in forming large object structures while keeping them manageable, decoupled, and easy to work with. One such pattern is the **Facade Pattern**, which simplifies complex systems by providing a unified interface. Let's dive deeper.

## Facade Pattern

The **Facade Pattern** is a **structural design pattern** that provides a simplified, unified interface to a complex subsystem or group of classes.

It acts as a **single entry point** for clients to interact with the system, hiding the underlying complexity and making the system easier to use.

### Real-Life Analogy

Think of Manual vs. Automatic Car:

- **Complex Subsystem (Manual Car):** Driving a manual car requires intricate knowledge of multiple components (clutch, gear shifter, accelerator) and their precise coordination to shift gears and drive. It's complex and requires the driver to manage many interactions.
- **Facade (Automatic Car):** An automatic car acts as a facade. It provides a simplified interface (e.g., "Drive," "Reverse," "Park") to the complex underlying mechanics of gear shifting. The driver (client) no longer needs to manually coordinate the clutch and gears; the automatic transmission handles these complexities internally, making driving much easier.

In short, the **manual car exposes the complexity**, while the **automatic car (the facade) simplifies it** for the user

### Problem It Solves

It solves the problem of **dealing with complex subsystems** by hiding the complexities behind a single, unified interface. **For example, imagine a movie ticket booking system with:**

- PaymentService
- SeatReservationService
- NotificationService
- LoyaltyPointsService
- TicketService

Instead of making the client interact with all of these directly, the Facade Pattern provides a **single class** like `MovieBookingFacade`, which internally coordinates all the services.

---

### Real-Life Coding Example

Imagine you're developing a movie ticket booking application, like BookMyShow. Let's first take a look at a poorly structured approach to implementing the booking functionality.

#### The Bad Way (Without Using Facade pattern)

```
#include <iostream>
```

```
using namespace std;
```

```
// Service class responsible for handling payments
```

```
class PaymentService {

public:

    void makePayment(string accountId, double amount) {

        cout << "Payment of ₹" << amount << " successful for account " <<
accountId << endl;

    }

};


// Service class responsible for reserving seats

class SeatReservationService {

public:

    void reserveSeat(string movieId, string seatNumber) {

        cout << "Seat " << seatNumber << " reserved for movie " << movieId <<
endl;

    }

};


// Service class responsible for sending notifications

class NotificationService {

public:

    void sendBookingConfirmation(string userEmail) {

        cout << "Booking confirmation sent to " << userEmail << endl;

    }

};
```

```
    }  
};  
  
// Service class for managing loyalty/reward points  
class LoyaltyPointsService {  
public:  
    void addPoints(string accountId, int points) {  
        cout << points << " loyalty points added to account " << accountId <<  
endl;  
    }  
};
```

```
// Service class for generating movie tickets  
class TicketService {  
public:  
    void generateTicket(string movieId, string seatNumber) {  
        cout << "Ticket generated for movie " << movieId << ", Seat: " <<  
seatNumber << endl;  
    }  
};
```

```
// Client Code
```

```
int main() {  
  
    // Booking a movie ticket manually (without a facade)  
  
    // Step 1: Make payment  
    PaymentService paymentService;  
    paymentService.makePayment("user123", 500);  
  
    // Step 2: Reserve seat  
    SeatReservationService seatReservationService;  
    seatReservationService.reserveSeat("movie456", "A10");  
  
    // Step 3: Send booking confirmation via email  
    NotificationService notificationService;  
    notificationService.sendBookingConfirmation("user@example.com");  
  
    // Step 4: Add loyalty points to user's account  
    LoyaltyPointsService loyaltyPointsService;  
    loyaltyPointsService.addPoints("user123", 50);  
  
    // Step 5: Generate the ticket  
    TicketService ticketService;
```

```
ticketService.generateTicket("movie456", "A10");
```

```
return 0;
```

```
}
```

```
// Service class responsible for handling payments
```

```
class PaymentService {
```

```
    public void makePayment(String accountId, double amount) {
```

```
        System.out.println("Payment of ₹" + amount + " successful for account  
" + accountId);
```

```
    }
```

```
}
```

```
// Service class responsible for reserving seats
```

```
class SeatReservationService {
```

```
    public void reserveSeat(String movieId, String seatNumber) {
```

```
        System.out.println("Seat " + seatNumber + " reserved for movie " +  
movieId);
```

```
    }
```

```
}
```

```
// Service class responsible for sending notifications
```

```
class NotificationService {  
  
    public void sendBookingConfirmation(String userEmail) {  
  
        System.out.println("Booking confirmation sent to " + userEmail);  
  
    }  
  
}
```

// Service class for managing loyalty/reward points

```
class LoyaltyPointsService {  
  
    public void addPoints(String accountId, int points) {  
  
        System.out.println(points + " loyalty points added to account " +  
accountId);  
  
    }  
  
}
```

// Service class for generating movie tickets

```
class TicketService {  
  
    public void generateTicket(String movieId, String seatNumber) {  
  
        System.out.println("Ticket generated for movie " + movieId + ", Seat: " +  
seatNumber);  
  
    }  
  
}
```

```
// Client Code
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        // Booking a movie ticket manually (without a facade)
```

```
        // Step 1: Make payment
```

```
        PaymentService paymentService = new PaymentService();
```

```
        paymentService.makePayment("user123", 500);
```

```
        // Step 2: Reserve seat
```

```
        SeatReservationService seatReservationService = new  
SeatReservationService();
```

```
        seatReservationService.reserveSeat("movie456", "A10");
```

```
        // Step 3: Send booking confirmation via email
```

```
        NotificationService notificationService = new NotificationService();
```

```
        notificationService.sendBookingConfirmation("user@example.com");
```

```
        // Step 4: Add loyalty points to user's account
```

```
        LoyaltyPointsService loyaltyPointsService = new  
LoyaltyPointsService();
```

```
        loyaltyPointsService.addPoints("user123", 50);
```

```
// Step 5: Generate the ticket

TicketService ticketService = new TicketService();

ticketService.generateTicket("movie456", "A10");

}

}

# Service class responsible for handling payments

class PaymentService:

    def make_payment(self, account_id, amount):

        print(f"Payment of ₹{amount} successful for account {account_id}")

# Service class responsible for reserving seats

class SeatReservationService:

    def reserve_seat(self, movie_id, seat_number):

        print(f"Seat {seat_number} reserved for movie {movie_id}")

# Service class responsible for sending notifications

class NotificationService:

    def send_booking_confirmation(self, user_email):

        print(f"Booking confirmation sent to {user_email}")
```

**# Service class for managing loyalty/reward points**

**class LoyaltyPointsService:**

**def add\_points(self, account\_id, points):**

**print(f"{points} loyalty points added to account {account\_id}")**

**# Service class for generating movie tickets**

**class TicketService:**

**def generate\_ticket(self, movie\_id, seat\_number):**

**print(f"Ticket generated for movie {movie\_id}, Seat: {seat\_number}")**

**# Client Code**

**if \_\_name\_\_ == "\_\_main\_\_":**

**# Booking a movie ticket manually (without a facade)**

**# Step 1: Make payment**

**payment\_service = PaymentService()**

**payment\_service.make\_payment("user123", 500)**

**# Step 2: Reserve seat**

**seat\_reservation\_service = SeatReservationService()**

```
seat_reservation_service.reserve_seat("movie456", "A10")
```

```
# Step 3: Send booking confirmation via email
```

```
notification_service = NotificationService()
```

```
notification_service.send_booking_confirmation("user@example.com")
```

```
# Step 4: Add loyalty points to user's account
```

```
loyalty_points_service = LoyaltyPointsService()
```

```
loyalty_points_service.add_points("user123", 50)
```

```
# Step 5: Generate the ticket
```

```
ticket_service = TicketService()
```

```
ticket_service.generate_ticket("movie456", "A10")
```

While this code works, it's tightly coupled. The **Main** class (or client code) is **manually calling** each subsystem service in the correct order and with the correct parameters.

This leads to:

- **High complexity** for the client
- **Duplicate code** if you have to do this in multiple places

- **Violation of the Single Responsibility Principle** (the Main class knows too much)

This sets the stage for the Facade Pattern, which will encapsulate all these steps in one high-level method like `bookTicket()` and make the client code clean and readable.

#### Using Facade Pattern

```
#include <iostream>
```

```
using namespace std;
```

```
// Service class responsible for handling payments
```

```
class PaymentService {
```

```
public:
```

```
    void makePayment(string accountId, double amount) {
```

```
        cout << "Payment of ₹" << amount << " successful for account " << accountId << endl;
```

```
    }
```

```
};
```

```
// Service class responsible for reserving seats
```

```
class SeatReservationService {
```

```
public:
```

```
    void reserveSeat(string movieId, string seatNumber) {
```

```
        cout << "Seat " << seatNumber << " reserved for movie " << movieId << endl;
    }
};
```

```
// Service class responsible for sending notifications
```

```
class NotificationService {
public:
    void sendBookingConfirmation(string userEmail) {
        cout << "Booking confirmation sent to " << userEmail << endl;
    }
};
```

```
// Service class for managing loyalty/reward points
```

```
class LoyaltyPointsService {
public:
    void addPoints(string accountId, int points) {
        cout << points << " loyalty points added to account " << accountId << endl;
    }
};
```

```
// Service class for generating movie tickets
```

```
class TicketService {
```

```
public:
```

```
    void generateTicket(string movieId, string seatNumber) {
```

```
        cout << "Ticket generated for movie " << movieId << ", Seat: " <<  
seatNumber << endl;
```

```
    }
```

```
};
```

```
// ===== The MovieBookingFacade class =====
```

```
class MovieBookingFacade {
```

```
private:
```

```
    PaymentService paymentService;
```

```
    SeatReservationService seatReservationService;
```

```
    NotificationService notificationService;
```

```
    LoyaltyPointsService loyaltyPointsService;
```

```
    TicketService ticketService;
```

```
public:
```

```
    // Constructor initializes all the subsystem services
```

```
    MovieBookingFacade() {}
```

```
// Method providing a simplified interface for booking a movie ticket

void bookMovieTicket(string accountId, string movieId, string
seatNumber, string userEmail, double amount) {

    paymentService.makePayment(accountId, amount);

    seatReservationService.reserveSeat(movieId, seatNumber);

    ticketService.generateTicket(movieId, seatNumber);

    loyaltyPointsService.addPoints(accountId, 50);

    notificationService.sendBookingConfirmation(userEmail);


// Indicate successful completion of the entire booking process.

    cout << "Movie ticket booking completed successfully!" << endl;

}

};
```

```
// Client Code
```

```
int main() {

    // Booking a movie ticket manually (using facade)

    MovieBookingFacade movieBookingFacade;

    movieBookingFacade.bookMovieTicket("user123", "movie456", "A10",
"user@example.com", 500);
```

```
        return 0;
    }

    // Service class responsible for handling payments
    class PaymentService {

        public void makePayment(String accountId, double amount) {

            System.out.println("Payment of ₹" + amount + " successful for account " + accountId);

        }

    }

    // Service class responsible for reserving seats
    class SeatReservationService {

        public void reserveSeat(String movieId, String seatNumber) {

            System.out.println("Seat " + seatNumber + " reserved for movie " + movieId);

        }

    }

    // Service class responsible for sending notifications
    class NotificationService {

        public void sendBookingConfirmation(String userEmail) {
```

```
        System.out.println("Booking confirmation sent to " + userEmail);
    }
}
```

```
// Service class for managing loyalty/reward points
```

```
class LoyaltyPointsService {

    public void addPoints(String accountId, int points) {

        System.out.println(points + " loyalty points added to account " +
accountId);

    }

}
```

```
// Service class for generating movie tickets
```

```
class TicketService {

    public void generateTicket(String movieId, String seatNumber) {

        System.out.println("Ticket generated for movie " + movieId + ", Seat: " +
seatNumber);

    }

}
```

```
// ===== The MovieBookingFacade class =====
```

```
class MovieBookingFacade {

    private PaymentService paymentService;

    private SeatReservationService seatReservationService;

    private NotificationService notificationService;

    private LoyaltyPointsService loyaltyPointsService;

    private TicketService ticketService;


    // Constructor to initialize all the subsystem services.

    public MovieBookingFacade() {

        this.paymentService = new PaymentService();

        this.seatReservationService = new SeatReservationService();

        this.notificationService = new NotificationService();

        this.loyaltyPointsService = new LoyaltyPointsService();

        this.ticketService = new TicketService();

    }


    // Method providing a simplified interface for booking a movie ticket

    public void bookMovieTicket(String accountId, String movieId, String
seatNumber, String userEmail, double amount) {

        paymentService.makePayment(accountId, amount);

        seatReservationService.reserveSeat(movieId, seatNumber);

        ticketService.generateTicket(movieId, seatNumber);

    }

}
```

```
loyaltyPointsService.addPoints(accountId, 50);

notificationService.sendBookingConfirmation(userEmail);

// Indicate successful completion of the entire booking process.
System.out.println("Movie ticket booking completed successfully!");
}
}
```

```
// Client Code
```

```
class Main {

    public static void main(String[] args) {

        // Booking a movie ticket manually (using facade)

        MovieBookingFacade movieBookingFacade = new
MovieBookingFacade();

        movieBookingFacade.bookMovieTicket("user123", "movie456", "A10",
"user@example.com", 500);

    }

}
```

```
# Service class responsible for handling payments
```

```
class PaymentService:
```

```
def make_payment(self, account_id, amount):  
    print(f"Payment of ₹{amount} successful for account {account_id}")
```

**# Service class responsible for reserving seats**

```
class SeatReservationService:
```

```
    def reserve_seat(self, movie_id, seat_number):  
        print(f"Seat {seat_number} reserved for movie {movie_id}")
```

**# Service class responsible for sending notifications**

```
class NotificationService:
```

```
    def send_booking_confirmation(self, user_email):  
        print(f"Booking confirmation sent to {user_email}")
```

**# Service class for managing loyalty/reward points**

```
class LoyaltyPointsService:
```

```
    def add_points(self, account_id, points):  
        print(f"{points} loyalty points added to account {account_id}")
```

**# Service class for generating movie tickets**

```
class TicketService:
```

```
    def generate_ticket(self, movie_id, seat_number):
```

```
print(f"Ticket generated for movie {movie_id}, Seat: {seat_number}")
```

```
# ===== The MovieBookingFacade class =====
```

```
class MovieBookingFacade:
```

```
    # Constructor to initialize all the subsystem services.
```

```
    def __init__(self):
```

```
        self.payment_service = PaymentService()
```

```
        self.seat_reservation_service = SeatReservationService()
```

```
        self.notification_service = NotificationService()
```

```
        self.loyalty_points_service = LoyaltyPointsService()
```

```
        self.ticket_service = TicketService()
```

```
    # Method providing a simplified interface for booking a movie ticket
```

```
    def book_movie_ticket(self, account_id, movie_id, seat_number,  
user_email, amount):
```

```
        self.payment_service.make_payment(account_id, amount)
```

```
        self.seat_reservation_service.reserve_seat(movie_id, seat_number)
```

```
        self.ticket_service.generate_ticket(movie_id, seat_number)
```

```
        self.loyalty_points_service.add_points(account_id, 50)
```

```
        self.notification_service.send_booking_confirmation(user_email)
```

# Indicate successful completion of the entire booking process.

```
print("Movie ticket booking completed successfully!")
```

# Client Code

```
if __name__ == "__main__":
```

```
    # Booking a movie ticket manually (using facade)
```

```
    movie_booking_facade = MovieBookingFacade()
```

```
    movie_booking_facade.book_movie_ticket("user123", "movie456",  
    "A10", "user@example.com", 500)
```

### How Facade Pattern Solves the Issue

By introducing **MovieBookingFacade**, we:

- Provide a **simple, unified interface** (bookMovieTicket()).
  - **Hide the complexity** of internal service calls from the client.
  - **Reduce coupling**, so changes in internal services don't affect the client.
  - Centralize the **workflow logic**, making it easier to update and reuse.
- 

### When to use Facade Pattern?

You should use **Facade pattern** when:

- **Subsystems are complex:** This means there are too many classes and too many dependencies within the system you are trying to simplify.

- **You want to provide a simpler API for the outer world:** The Facade acts as a simplified entry point, hiding the complexity from clients.
  - **You want to reduce coupling between subsystems and client code:** By interacting with the facade, the client code becomes less dependent on the individual components of the subsystem.
  - **You want to layer your architecture cleanly:** The Facade helps in organizing the system into distinct layers, making it more modular and understandable.
- 

## Advantages

A few advantages of using the Facade Pattern are:

- **Lightweight coupling:** It reduces the dependencies between the client and the subsystem.
  - **Flexibility:** It allows the subsystem to evolve without impacting the client code.
  - **Simplifies client design:** Clients interact with a single, simplified interface instead of multiple complex objects.
  - **Promotes layered architecture:** It helps organize the system into distinct layers, improving maintainability and scalability.
  - **Better testability:** Individual subsystem components can be tested independently, and the facade itself can be tested for its orchestration logic.
- 

## Disadvantages

A few disadvantages of using the Facade Pattern are:

- **Fragile coupling:** If the facade itself changes frequently, it can still lead to ripple effects on client code.

- **Hidden complexity:** While it simplifies the client's view, the underlying complexity of the subsystem still exists, just hidden. This can make debugging or understanding the full flow more challenging for developers working on the subsystem.
- **Runtime errors:** Errors originating from the complex subsystem might be harder to diagnose when only interacting through the facade.
- **Difficult to trace:** Debugging can be more challenging as the facade adds another layer of indirection.
- **Violation of SRP (Single Responsibility Principle):** A facade might take on too many responsibilities if it orchestrates a very large and diverse set of operations, potentially becoming a "god object."

---

## Class Diagram



